Creative Assignment 1: Behavior Detection

I built a classifier that predicts whether an observation clip is coded OffTask = Y or N. Repeated five-fold cross-validation grouped by student, with all model selection nested inside the training students, gives a pooled Cohen's Kappa of 0.328 (mean fold Kappa 0.323). The final model is gradient boosting with 100 trees of depth 2 and a decision threshold of 0.40.

1. Data and Features

1.1 Exploring the data

ca1-dataset.csv contains 763 clips and 27 columns: two identifiers, the label and 24 numeric behavior summaries. There are no empty cells or duplicate rows. Only 34 clips (4.5%) are Y, so a detector that always predicts N reaches 95.5% accuracy with Kappa = 0.000. Kappa is therefore the main metric, supported by precision and recall for Y.

The clips come from 89 students (3 to 16 clips each, median 8), and the 34 Y clips belong to 22 students. Every validation split keeps all clips of a student together. The two prefixes in Unique-id have different observed Y rates (2.3% in 567 clips and 10.7% in 196 clips). This identifier encodes collection and session information, including dates and clip numbers, so I exclude it from prediction.

The two window timing columns separate the classes most clearly: Avgtimelast3SDnormed has a median of 2.12 for Y clips and −0.24 for N clips, and Avgtimelast5SDnormed 3.13 against −0.53 (univariate AUC 0.76 for both). Correctness columns such as Avgright show almost no univariate signal (AUC 0.50).

1.2 Feature screening

I use namea only to group clips by student during validation and exclude both identifiers from prediction. Nine columns are zero for every clip, and Avgnotright equals 1 − Avgright; I remove these ten columns. Fourteen raw inputs remain:

Table 1. Raw input features

Input group	Columns
Correctness	Avgright, Avgbug
Timing	Avgtime, AvgtimeSDnormed, Avgtimelast3SDnormed, Avgtimelast5SDnormed, Avgtimeperact-up
Error summaries	Avghowmanywrong-up, Avgwrongpct-up, Avg recent5wrong, Avgmanywrong-up
Activity summaries	Avgpchange, AvgPrev3Count-up, AvgPrev5Count-up

1.3 Preprocessing and added features

Two clips have Avgtime = -1, which I treat as missing along with the paired AvgtimeSDnormed values of -1. Within each training fold, preprocessing learns the medians of valid clips and uses them to replace these entries. It also adds three features calculated within each row:

Table 2. Added features and descriptive AUCs

Feature	Formula	Univariate AUC
TimeJump3	Avgtimelast3SDnormed − AvgtimeSDnormed	0.784
TimeJump5	Avgtimelast5SDnormed − AvgtimeSDnormed	0.781
TimePerActRatio	Avgtimeperact-up / max(Avgtime, 1)	0.568

TimeJump3 and TimeJump5 have the highest single-feature AUCs in this dataset. These features are numerical contrasts between the supplied timing summaries. They follow the course example that slow actions can indicate that a student has stopped using the software. The AUC table describes patterns in the full dataset; the held-out evaluation appears in Section 3. I use the SD-normalized columns as supplied. Applying the detector to new logs requires the same feature definitions, measurements available by the end of each clip, and normalization references estimated from training or earlier data.

2. Model Building and Validation

Each candidate is a scikit-learn pipeline containing the preprocessing step and a classifier; logistic regression also standardizes its inputs. The search covers 10 configurations from four algorithm families and 11 decision thresholds (0.10 to 0.60 in steps of 0.05).

Table 3. Candidate algorithms and parameter settings

Algorithm	Settings considered
Logistic regression	C = 0.1 or 1; balanced class weights
Decision tree	Depth = 3 or 5; minimum leaf size = 10; balanced class weights
Random forest	200 trees; leaf size = 3, or leaf size = 10 with balanced_subsample weights
Gradient boosting	Depth = 2 or 3; 100 or 200 trees; learning rate = 0.05; subsample = 0.8

The outer evaluation uses five folds grouped by namea, repeated with seeds 0 to 4 (25 outer folds). In every outer fold, the training students enter an inner search with four grouped folds and two repeats. For each candidate and threshold, the inner search pools the held-out predictions within each repeat, calculates Kappa and averages the two values; the highest mean selects the model and threshold. The selected pipeline is fitted on the outer training students and evaluated on the outer test students, who take no part in fitting or selection. The code checks student separation in all 25 outer and 200 inner splits.

The analysis uses scikit-learn 1.8.0, which includes a correction to shuffled StratifiedGroupKFold. The notebook fixes this version, records package versions, and uses fixed random seeds to support reproduction of the results.

3. Results and Interpretation for New Students

Each repeat supplies one held-out prediction per clip. Pooled Kappa is calculated across the 763 predictions within a repeat and averaged over the five repeats; mean fold Kappa averages the 25 outer-fold values.

Table 4. Performance on held-out students

Measure	Result
Pooled Kappa	0.328 (SD across repeats 0.067; range 0.218 to 0.383)
Mean fold Kappa	0.323 (SD 0.160)
AUC	0.777
Precision for Y	0.413
Recall for Y	0.318
Accuracy	0.949
Always predict N	Kappa 0.000; accuracy 0.955

The positive Kappa (0.328) indicates agreement with the observed labels beyond chance on held-out students. The detector identifies 31.8% of off-task clips, and 41.3% of its off-task flags are correct. This means that it misses about two-thirds of observed off-task clips, and more than half of its flags are false alarms. Averaged across repeats, it correctly identifies 10.8 of the 34 Y clips and produces 15.6 false alarms among the 729 N clips.

Within each outer fold, I also recorded the best candidate and threshold within each algorithm family, chosen by the same inner rule. These rows compare the algorithms; the full procedure alone builds the final model.

Table 5. Algorithm comparison under nested cross-validation

Algorithm family	Pooled Kappa	Precision (Y)	Recall (Y)
Logistic regression	0.218	0.187	0.471
Decision tree	0.116	0.108	0.594
Random forest	0.321	0.362	0.347
Gradient boosting	0.334	0.495	0.282
Full procedure	**0.328**	**0.413**	**0.318**

The tree ensembles reach the highest Kappa. Logistic regression and the decision tree flag many N clips (precision below 0.19), which lowers their Kappa. The full procedure selected a random forest or a gradient boosting model in all 25 outer folds.

4. Final Model and Findings

For the final detector, I apply the same inner selection to all 763 clips and fit the selected pipeline on the full data set. It selects gradient boosting with 100 trees, depth 2, learning rate 0.05 and subsample 0.8, and predicts Y when its score is at least 0.40. Its preprocessing uses full-data medians of 9.667 for Avgtime and −0.121 for AvgtimeSDnormed. The expected performance for new students is the nested estimate in Table 4.

Table 6. Feature importance in the final model

Feature	Impurity importance
TimePerActRatio	0.256
TimeJump5	0.247
TimeJump3	0.152
Avgtime	0.133
Avgtimelast3SDnormed	0.081
Avgtimeperact-up	0.060

The final model gives the highest feature importance values to timing inputs, led by the three added features. Together with the single-feature AUCs, this supports timing summaries as useful predictors in this dataset. These importance values describe the fitted tree splits, and correlated inputs can share importance.

5. Scope and Submitted Files

The validation estimates performance for additional students with comparable tutor logs and observation procedures. The 34 positive clips limit the precision of this estimate.

Table 7. Submitted files

File	Content
Mengchen_Li_6191_CA1.pdf	This report
Colab notebook and the link (Mengchen_Li_6191_CA1.ipynb) Open Colab notebook	Complete code with comments and outputs
Mengchen_Li_6191_CA1_modeling_data.csv	Data entered into the model: identifiers, label and the 14 raw inputs

Mengchen_Li_6191_CA1_final_model.joblib	Final fitted classifier, preprocessing medians, feature names, threshold and validation summary

6. AI Use Statement

I used generative AI tools (Claude and ChatGPT) as assistants in this assignment. They helped me write and debug the Python code, check and interpret the code outputs, and draft and revise parts of this report. I reviewed the code, ran the complete notebook in Google Colab to produce all reported results, and take responsibility for the analysis and the final text.